

Material dezvoltat pentru examen IA - Invatare automata

Teorie + rezolvare model + exemple explicate

Bazat pe modelul din poza si pe cursurile/laboratoarele incarcate

Cum folosesti documentul:

- Invata intai grila si rezolvarile de la final: ele iti arata exact ce intreaba profesorul.
- Apoi citeste teoria asociata: SVM, kernel, normalizare, activari, retele neuronale, optimizare si metrici.
- Pentru problemele deschise, invata sabloanele: "calculez scorul", "verific gradientul", "propun neuronul", "demonstrez separarea".

Contents

1	Ce pare sa intre dupa modelul de examen	4
1.1	Ordinea recomandata de invatare	4
2	Rezolvarea rapida a grilei din poza	5
3	SVM si marginile maxime	5
3.1	Ideea clasificatorului liniar	5
3.2	SVM hard-margin	5
3.3	SVM soft-margin si variabila ξ	5
3.4	Parametrul C	6
3.5	Hinge loss	6
4	Metode kernel	7
4.1	De ce avem nevoie de kerneluri?	7
4.2	Kerneluri importante	7
4.3	Operatii valide cu kerneluri	7
4.4	Exemplu de calcul: string kernel cu 2-grame	8
5	Normalizarea datelor	9
5.1	Standardizarea pe dimensiuni	9
5.2	Normalizarea L2 pe exemple	9
5.3	Batch Normalization	9
5.4	Batch de un singur exemplu	9
6	Functii de activare	10
6.1	Sigmoid	10
6.2	Tanh	10
6.3	ReLU	10
6.4	Leaky ReLU	10
6.5	Maxout	11
6.6	Identitate utila din grila	11
7	Optimizare: GD, SGD si momentum	12
7.1	Gradient descent	12
7.2	SGD si mini-batch	12
7.3	SGD cu momentum	12
7.4	Ce se intampla daca $\mu = 1$?	12
7.5	Rata de invatare	13
8	Erori in invatarea automata	13

8.1 Eroare empirica	13
8.2 Eroare de generalizare	13
8.3 Cand $n \rightarrow \infty$	13
8.4 Descompunerea erorii	13
8.5 Bias si variance	13
9 Retele neuronale	14
9.1 De ce nu ajunge un clasificator liniar?	14
9.2 Conditia din grila	14
9.3 Model cu doua straturi	14
9.4 Dead ReLU	14
10 Metrici de evaluare	15
10.1 Acuratete	15
10.2 Matrice de confuzie	15
10.3 Precision, Recall, F1	15
10.4 Kendall Tau	15
10.5 Exemplu din grila	15
11 Problema 11 din model: neuron ReLU si Maxout	16
11.1 a) Aplicarea neuronului pe date	16
11.2 b) Ce problema apare la optimizare?	16
11.3 c) Putem construi un neuron Maxout cu eroare 0?	17
12 Problema 12 din model: normalizare L2 si separare	18
12.1 a) Obiectul geometric	18
12.2 b) Separarea punctelor cu prima coordonata pozitiva	18
13 Naive Bayes - recap util	19
13.1 Regula Bayes	19
13.2 Presupunerea Naive Bayes	19
13.3 Ce faci daca apare o valoare nevazuta?	19
14 KNN - recap util	19
14.1 Algoritmul	19
14.2 Distanțe	20
14.3 Efectul lui k	20
15 Regresie liniara si Ridge	20
15.1 Regresie liniara	20
15.2 Ridge	20
15.3 Lasso	20

16	Mini-exercitii tip examen, rezolvate	21
16.1	Exercitiul 1 - SVM soft margin	21
16.2	Exercitiul 2 - BatchNorm cu batch de un exemplu	21
16.3	Exercitiul 3 - Kendall Tau	21
16.4	Exercitiul 4 - Retele pentru XOR	21
16.5	Exercitiul 5 - Maxout pentru $ x_1 - x_3 $	21
17	Fisa de memorat inainte de examen	22
18	Surse folosite	24
A	Modelul din poza - transcriere scurta a temelor	24

1 Ce pare sa intre dupa modelul de examen

Modelul din poza are doua tipuri de cerinte:

1. **Grila teoretica rapida:** SVM soft-margin, batch normalization, functii de activare, string kernels, functii kernel valide, RBF, SGD cu momentum, retele neuronale, eroare empirica/generalizare, Kendall Tau.
2. **Probleme scurte de calcul/demonstratie:** neuron ReLU pe o multime mica, problema de optimizare cu ReLU mort, constructie cu maxout, normalizare L2 si separare liniara pe sfera.

De retinut. Daca modelul acesta este reprezentativ, trebuie sa poti rezolva formule scurte si sa justifici in 2-5 randuri. Nu este suficient sa stii definitii; trebuie sa stii de ce o varianta e corecta si de ce celelalte sunt gresite.

1.1 Ordinea recomandata de invatare

Pas	Ce inveti
1	SVM + kernel: soft margin, variabila ξ , kernel linear/RBF/polinomial/intersectie, string kernel, operatii valide cu kerneluri.
2	Normalizari: standardizare, normalizare L1/L2, batch normalization.
3	Functii de activare: sigmoid, tanh, ReLU, Leaky ReLU, Maxout; cand omoara gradientul.
4	Optimizare: gradient descent, SGD, mini-batch, momentum, learning rate, eroare de optimizare.
5	Retele neuronale: strat ascuns, activare neliniara, problema XOR, maxout.
6	Evaluare: eroare empirica, eroare de generalizare, bias/variance, Kendall Tau, acuratete, confusion matrix.
7	Rezolvi modelul complet fara sa te uiti la teorie.

2 Rezolvarea rapida a grilei din poza

Nr.	Raspuns	Explicatie scurta
1	D, ξ	In SVM soft-margin, variabilele slack ξ_i permit incalcarea marginii / clasificari gresite controlate.
2	B	BatchNorm pe un batch cu un singur exemplu: media pe fiecare dimensiune este chiar valoarea exemplului, dispersia este 0, iar valoarea normalizata devine 0, cu ϵ in numitor.
3	A	$\frac{e^x - e^{-x}}{e^x + e^{-x}} = \tanh(x)$, deci $\max(0, \tanh(x)) = \text{ReLU}(\tanh(x))$.
4	B, 7	Se iau 2-gramele unice comune intre cele doua siruri, incluzand spatiile. Intersectia are 7 elemente.
5	D	Niciuna nu este garantat kernel valid in forma data. Operatiile valide cer simetrie si pozitiv semidefinitate; suma/produsul sunt ok doar daca termenii sunt kerneluri valide pe aceleasi doua argumente.
6	B	Kernelul RBF foloseste distanta L2: $K(u, v) = \exp(-\gamma \ u - v\ ^2)$.
7	D	La momentum $\mu = 1$ nu mai exista amortizare. Viteza acumuleaza gradientii si algoritmul poate diverge/oscila.
8	A	Pentru date nelinier separabile, e nevoie de cel putin un strat ascuns cu activare neliniara.
9	D	Cand $n \rightarrow \infty$, eroarea empirica estimeaza eroarea de generalizare, presupunand date i.i.d.
10	C, 0.6	Sunt 10 perechi. Avem $P = 8$ concordante si $Q = 2$ discordante, deci $\tau = (P - Q)/10 = 0.6$.

3 SVM si marginile maxime

3.1 Ideea clasificatorului liniar

Pentru clasificare binara, un clasificator liniar decide cu:

$$f(x) = w^T x + b.$$

Granita de decizie este hiperplanul:

$$w^T x + b = 0.$$

Daca $f(x) > 0$, exemplul este in clasa +1; daca $f(x) < 0$, este in clasa -1.

3.2 SVM hard-margin

Cand datele sunt liniar separabile perfect, SVM cauta hiperplanul cu **margină maximă**. Conditia standard este:

$$y_i(w^T x_i + b) \geq 1, \quad \forall i.$$

Marginea geometrica este proportionala cu $1/\|w\|$, deci maximizarea marginii inseamna minimizarea lui $\|w\|^2$.

3.3 SVM soft-margin si variabila ξ

Cand datele nu sunt liniar separabile, introducem variabile slack:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Interpretare:

- $\xi_i = 0$: punctul respecta marginea;
- $0 < \xi_i < 1$: punctul este in interiorul marginii, dar inca pe partea corecta;
- $\xi_i \geq 1$: punctul poate fi clasificat gresit.

Funcția optimizată este aproximativ:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i.$$

Parametrul C controlează cât de sever pedepsim erorile.

Tip examen. Dacă întrebarea spune “puncte neliniar separabile” și “ignorând anumite puncte”, te gândești imediat la soft-margin și la ξ , nu la w și nu la γ .

3.4 Parametrul C

- C mare: modelul evită greselile pe train; margine mai mică; risc de overfitting.
- C mic: modelul acceptă mai multe greseli; margine mai mare; risc de underfitting.

3.5 Hinge loss

Pentru un exemplu:

$$L_i = \max(0, 1 - y_i(w^T x_i + b)).$$

Aceasta este exact ideea din SVM: dacă exemplul este corect și suficient de departe de hiperplan, pierderea este 0; altfel este pozitivă.

4 Metode kernel

4.1 De ce avem nevoie de kerneluri?

O metoda liniara nu poate rezolva probleme precum XOR in spatiul original. Ideea kernel este:

1. transformam implicit datele intr-un spatiu mai mare;
2. cautam un hiperplan liniar in spatiul transformat;
3. in spatiul original, granita devine neliniara.

Un kernel este o functie de similaritate:

$$K(x, y) = \langle \phi(x), \phi(y) \rangle,$$

unde ϕ este transformarea implicita.

4.2 Kerneluri importante

Kernel	Formula	Cand apare
Linear	$K(x, y) = x^T y$	date aproximativ liniar separabile
Polinomial	$K(x, y) = (x^T y + c)^d$	interactiuni polinomiale intre trasaturi
RBF / Gaussian	$K(x, y) = \exp(-\gamma \ x - y\ ^2)$	date neliniare; foloseste distanta L2
Intersectie	$K(x, y) = \sum_i \min(x_i, y_i)$	histograme, frecvente, imagini/text
String kernel	numara n -grame comune	siruri de caractere, texte

De retinut. Daca intrebarea cere ce transforma distanta L_2 in kernel, raspunsul este RBF: $\exp(-\gamma \|x - y\|^2)$.

4.3 Operatii valide cu kerneluri

Daca k_1 si k_2 sunt kerneluri valide, atunci sunt kerneluri valide:

$$k(x, y) = k_1(x, y) + k_2(x, y),$$

$$k(x, y) = a k_1(x, y), \quad a > 0,$$

$$k(x, y) = k_1(x, y)k_2(x, y),$$

$$k(x, y) = f(x)f(y),$$

$$k(x, y) = k_1(\phi(x), \phi(y)).$$

Capcana. Nu orice combinatie scrisa cu k_1, k_2, k_3 este automat kernel. Trebuie sa fie simetrica in argumente si pozitiv semidefinita. Daca apare o transformare aplicata doar lui x sau doar lui y , fara simetrie, nu poti garanta ca este kernel.

4.4 Exemplu de calcul: string kernel cu 2-grame

Pentru string kernel bazat pe biti de prezenta, fiecare sir devine multimea 2-gramelor unice. Valoarea kernelului este numarul de 2-grame comune.

Sirurile din grila:

$$s = \text{"ana are mere"}, \quad t = \text{"ioana are banane"}.$$

Consideram si spatiile, notate aici cu "_".

2-grame unice pentru s :

$$\{an, na, a_, _a, ar, re, e_, _m, me, er\}.$$

2-grame unice pentru t :

$$\{io, oa, an, na, a_, _a, ar, re, e_, _b, ba, ne\}.$$

Intersectia este:

$$\{an, na, a_, _a, ar, re, e_ \}.$$

Deci valoarea kernelului este:

$$K(s, t) = 7.$$

5 Normalizarea datelor

5.1 Standardizarea pe dimensiuni

Daca X este matricea de date, cu exemple pe linii si trasaturi pe coloane, standardizarea face:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

Fiecare trasatura ajunge sa aiba medie 0 si deviatie standard 1 pe setul de antrenare.

5.2 Normalizarea L2 pe exemple

Pentru un exemplu $x \in \mathbb{R}^d$:

$$x' = \frac{x}{\|x\|_2}, \quad \|x'\|_2 = 1.$$

In $\mathbb{R}^3$, toate punctele nenule ajung pe suprafata sferei unitate:

$$S^2 = \{x \in \mathbb{R}^3 : \|x\|_2 = 1\}.$$

Tip examen. Daca te intreaba “pe ce obiect geometric se proiecteaza punctele din $\mathbb{R}^3$ dupa normalizare L2?”, raspunsul este: pe sfera unitate, adica suprafata sferei de raza 1 centrata in origine.

5.3 Batch Normalization

BatchNorm normalizeaza activari pe mini-batch. Pentru o dimensiune j :

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij}, \quad \sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2,$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}.$$

Apoi, in forma completa, se aplica parametri invatabili:

$$y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

5.4 Batch de un singur exemplu

Daca batch-ul are un singur exemplu, pentru fiecare dimensiune:

$$\mu_j = x_{1j}, \quad \sigma_j^2 = 0.$$

Deci:

$$\hat{x}_{1j} = \frac{x_{1j} - x_{1j}}{\sqrt{0 + \varepsilon}} = 0.$$

Prin urmare, vectorul normalizat devine vectorul zero, iar norma lui devine 0.

Capcana. Nu alegi “nu se poate aplica pentru ca se imparte la 0”. In formula exista ε tocmai pentru stabilitate numerica.

6 Functii de activare

6.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Proprietati:

- transforma in intervalul $(0, 1)$;
- utila pentru probabilitati binare;
- are probleme de saturare: pentru x foarte mare sau foarte mic, gradientul este aproape 0.

Derivata:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

6.2 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Proprietati:

- transforma in intervalul $[-1, 1]$;
- este centrata in zero;
- tot se poate satura, deci poate omori gradientii.

6.3 ReLU

$$\text{ReLU}(x) = \max(0, x).$$

Derivative informal:

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases}$$

La $x = 0$ derivata este nedefinita; in practica se alege de obicei 0 sau 1.

Avantaje:

- simpla si rapida;
- nu se satureaza pentru valori pozitive;
- antreneaza retele adanci mai bine decat sigmoid/tanh.

Problema:

- pentru intrari negative, gradientul este 0;
- daca neuronul ajunge sa dea mereu valori negative inainte de ReLU, nu se mai actualizeaza: **dead ReLU**.

6.4 Leaky ReLU

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \quad \alpha > 0 \text{ mic.}$$

Nu omoara complet gradientul pe partea negativa.

6.5 Maxout

Un neuron Maxout calculeaza maximul dintre mai multe functii liniare:

$$f(x) = \max_{j=1}^k (w_j^T x + b_j).$$

Este neliniar, liniar pe portiuni, nu se satureaza si poate generaliza ReLU/Leaky ReLU.

6.6 Identitate utila din grila

$$\max\left(0, \frac{e^x - e^{-x}}{e^x + e^{-x}}\right) = \max(0, \tanh(x)) = \text{ReLU}(\tanh(x)).$$

7 Optimizare: GD, SGD si momentum

7.1 Gradient descent

Avem o functie de pierdere $L(W)$. Vrem:

$$W^* = \arg \min_W L(W).$$

Actualizarea standard:

$$W \leftarrow W - \eta \nabla_W L(W),$$

unde η este rata de invatare.

7.2 SGD si mini-batch

In loc sa calculam gradientul pe tot setul de antrenare, folosim un mini-batch:

$$W \leftarrow W - \eta \nabla_W L_{batch}(W).$$

Avantaje:

- mai rapid pe seturi mari;
- zgomotul gradientului poate ajuta optimizarea;
- este standard in retele neuronale.

7.3 SGD cu momentum

Momentum introduce o viteza:

$$v_t = \mu v_{t-1} - \eta \nabla_W L(W_t),$$

$$W_{t+1} = W_t + v_t.$$

Interpretare:

- μ controleaza cat din viteza trecuta pastram;
- valori uzuale: 0.9, 0.95, 0.99;
- ajuta pe directii cu panta lina si reduce zig-zag-ul pe directii abrupte.

7.4 Ce se intampla daca $\mu = 1$?

Daca $\mu = 1$, viteza devine:

$$v_t = v_{t-1} - \eta \nabla_W L(W_t).$$

Nu mai exista amortizare. Gradientii trecuti se acumuleaza prea mult, iar algoritmul poate oscila sau diverge.

Tip examen. La grila, "momentum egal cu 1" nu inseamna SGD standard. SGD standard este cazul $\mu = 0$. Pentru $\mu = 1$, raspunsul bun este ca algoritmul diverge / devine instabil.

7.5 Rata de invatare

- prea mare: sari peste minim, pierderea explodeaza sau oscileaza;
- prea mica: invata foarte lent;
- potrivita: loss scade constant.

8 Erori in invatarea automata

8.1 Eroare empirica

Eroarea empirica este media pierderii pe un set finit de date:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n L(f(x_i), y_i).$$

8.2 Eroare de generalizare

Eroarea de generalizare este eroarea asteptata pe date noi, din distributia reala:

$$R(f) = \mathbb{E}_{(X,Y) \sim P}[L(f(X), Y)].$$

8.3 Cand $n \rightarrow \infty$

Daca datele sunt i.i.d., atunci prin legea numerelor mari:

$$\hat{R}(f) \rightarrow R(f).$$

Deci eroarea empirica devine o estimare a erorii de generalizare.

8.4 Descompunerea erorii

In curs apar trei tipuri de erori:

- **eroarea de modelare**: spatiul de ipoteze nu poate reprezenta realitatea;
- **eroarea de estimare**: avem date finite si estimam imperfect;
- **eroarea de optimizare**: nu gasim minimul functiei obiectiv.

8.5 Bias si variance

- Bias mare: model prea simplu, underfitting.
- Variance mare: model prea sensibil la date, overfitting.

9 Retele neuronale

9.1 De ce nu ajunge un clasificator liniar?

Un clasificator liniar separa datele printr-un hiperplan. Pentru probleme neliniar separabile, precum XOR, nu exista o singura dreapta/plan care separa corect clasele.

O retea neuronală cu strat ascuns poate construi granite neliniare dacă folosește activare neliniară.

9.2 Conditia din grila

Pentru ca o retea sa invete date neliniar separabile, trebuie:

cel putin un strat ascuns + activare neliniara.

Dacă toate activările sunt liniare, compunerea straturilor liniare ramane tot o funcție liniară.

Capcana. Un strat softmax la ieseire nu este suficient pentru a învăța separări neliniare. Softmax normalizează scoruri; neliniaritatea utilă trebuie să fie în reprezentarea ascunsă.

9.3 Model cu doua straturi

Un model simplu:

$$h = \phi(W_1x + b_1),$$

$$s = W_2h + b_2.$$

Dacă ϕ este ReLU/tanh/sigmoid/Maxout, modelul poate învăța frontiere neliniare.

9.4 Dead ReLU

Dacă pentru toate exemplele:

$$z = w^T x + b < 0,$$

atunci:

$$\text{ReLU}(z) = 0,$$

și gradientul prin ReLU este 0. Neuronul nu se mai actualizează. Aceasta este problema de **dead ReLU**.

Soluții:

- bias initial ușor pozitiv;
- learning rate mai mic;
- Leaky ReLU;
- BatchNorm;
- initializare potrivită.

10 Metrici de evaluare

10.1 Acuratete

$$\text{Acc} = \frac{\# \text{predictii corecte}}{\# \text{exemple}}.$$

10.2 Matrice de confuzie

Pe linii pui clasele reale, pe coloane clasele prezise. Elementul C_{ij} inseamna:

$$C_{ij} = \# \text{exemple din clasa reala } i \text{ prezise ca } j.$$

10.3 Precision, Recall, F1

Pentru binar:

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}.$$

10.4 Kendall Tau

Kendall Tau masoara cat de bine se potrivesc doua ordonari. Pentru fiecare pereche de exemple:

- concordanta: ordinea reala si ordinea prezisa coincid;
- discordanta: ordinea reala si ordinea prezisa sunt inverse.

Formula:

$$\tau = \frac{P - Q}{\binom{n}{2}},$$

unde P este numarul de perechi concordante, iar Q numarul de perechi discordante.

10.5 Exemplu din grila

Etichete:

$$[0.1, 0.2, 0.3, 0.4, 0.5]$$

Predictii:

$$[0.4, 0.2, 0.6, 1.0, 0.8].$$

Ordinea reala este:

$$1 < 2 < 3 < 4 < 5.$$

Ordinea dupa predictii este:

$$2 < 1 < 3 < 5 < 4.$$

Sunt 10 perechi. Perechile discordante sunt (1, 2) si (4, 5), deci:

$$P = 8, \quad Q = 2.$$

$$\tau = \frac{8 - 2}{10} = 0.6.$$

11 Problema 11 din model: neuron ReLU si Maxout

Se da:

$$S = \{([1, 1, 0], 1), ([0, 1, 0], 0), ([0, 1, 1], 1), ([1, 1, 1], 0)\}.$$

Neuronul are:

$$w = [0, -1, 0], \quad b = 0.5,$$

si activare ReLU:

$$\hat{y} = \text{ReLU}(w^T x + b).$$

11.1 a) Aplicarea neuronului pe date

Pentru orice exemplu din S , a doua componenta este $x_2 = 1$. Deci:

$$w^T x + b = 0 \cdot x_1 - 1 \cdot x_2 + 0 \cdot x_3 + 0.5 = -1 + 0.5 = -0.5.$$

Prin ReLU:

$$\text{ReLU}(-0.5) = 0.$$

Deci toate exemplele primesc output 0.

Exemplu	Eticheta	$z = w^T x + b$	$\text{ReLU}(z)$
[1, 1, 0]	1	-0.5	0
[0, 1, 0]	0	-0.5	0
[0, 1, 1]	1	-0.5	0
[1, 1, 1]	0	-0.5	0

Daca interpretam iesirea 0 ca clasa 0, atunci exemplele cu eticheta 0 sunt corecte, iar cele cu eticheta 1 sunt gresite.

11.2 b) Ce problema apare la optimizare?

Apare problema de **dead ReLU**. Pentru toate exemplele:

$$z = -0.5 < 0.$$

Prin urmare, derivata ReLU este 0:

$$\text{ReLU}'(z) = 0.$$

Gradientul nu trece prin neuron si ponderile nu se actualizeaza eficient.

O valoare buna pentru bias este, de exemplu:

$$b = 1.5.$$

Atunci:

$$z = -1 + 1.5 = 0.5 > 0.$$

Deci ReLU este activa:

$$\text{ReLU}(0.5) = 0.5,$$

iar gradientul poate trece.

De retinut. Noua valoare a lui b nu este propusa ca sa clasifice perfect setul; este propusa ca sa rezolve problema de optimizare: neuronul nu mai este mort.

11.3 c) Putem construi un neuron Maxout cu eroare 0?

Da. Observam ca eticheta este 1 exact cand $x_1 \neq x_3$:

$$[1, 1, 0] \rightarrow 1,$$

$$[0, 1, 1] \rightarrow 1,$$

$$[0, 1, 0] \rightarrow 0,$$

$$[1, 1, 1] \rightarrow 0.$$

Adica avem o forma XOR intre x_1 si x_3 .

Construim Maxout cu doua componente liniare:

$$f_1(x) = x_1 - x_3,$$

$$f_2(x) = x_3 - x_1.$$

Neuronul Maxout:

$$\hat{y} = \max(f_1(x), f_2(x)) = |x_1 - x_3|.$$

Verificare:

x	f_1	f_2	max	y
[1, 1, 0]	1	-1	1	1
[0, 1, 0]	0	0	0	0
[0, 1, 1]	-1	1	1	1
[1, 1, 1]	0	0	0	0

Deci eroarea este 0.

12 Problema 12 din model: normalizare L2 si separare

12.1 a) Obiectul geometric

Pentru orice punct nenul $x \in \mathbb{R}^3$, normalizarea L2 este:

$$\tilde{x} = \frac{x}{\|x\|_2}.$$

Atunci:

$$\|\tilde{x}\|_2 = 1.$$

Deci toate punctele se proiecteaza pe:

$$S^2 = \{x \in \mathbb{R}^3 : x_1^2 + x_2^2 + x_3^2 = 1\}.$$

Aceasta este **sfera unitate** in $\mathbb{R}^3$, mai exact suprafata sferei.

12.2 b) Separarea punctelor cu prima coordonata pozitiva

Vrem sa separăm punctele normalizate cu:

$$x_1 > 0$$

de toate celelalte.

Putem folosi un neuron liniar cu:

$$w = (1, 0, 0), \quad b = 0.$$

Scorul este:

$$f(x) = w^T x + b = x_1.$$

Decizia:

$$\hat{y} = \begin{cases} 1, & x_1 > 0, \\ 0, & x_1 \leq 0. \end{cases}$$

Hiperplanul de separare este:

$$x_1 = 0.$$

Acesta este planul yz , care separa emisfera cu $x_1 > 0$ de restul.

Tip examen. Daca in enunt scrie doar " $x > 0$ " pentru puncte din $\mathbb{R}^3$, interpreteaza ca prima coordonata este pozitiva. Scrie explicit "presupun ca se refera la coordonata x_1 ".

13 Naive Bayes - recap util

13.1 Regula Bayes

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}.$$

Pentru clasificare, $P(x)$ este acelasi pentru toate clasele, deci:

$$\hat{c} = \arg \max_c P(x|c)P(c).$$

13.2 Presupunerea Naive Bayes

Daca $x = (x_1, \dots, x_d)$:

$$P(x|c) = \prod_{j=1}^d P(x_j|c).$$

Decizia devine:

$$\hat{c} = \arg \max_c P(c) \prod_{j=1}^d P(x_j|c).$$

In practica folosim logaritmi:

$$\hat{c} = \arg \max_c \left(\log P(c) + \sum_{j=1}^d \log P(x_j|c) \right).$$

13.3 Ce faci daca apare o valoare nevazuta?

Folosesti smoothing, de exemplu Laplace:

$$P(x_j = v|c) = \frac{\text{count}(x_j = v, c) + \alpha}{\text{count}(c) + \alpha|V_j|}.$$

Fara smoothing, probabilitatea poate deveni 0 si toata clasa este anulata.

14 KNN - recap util

14.1 Algoritmul

Pentru un exemplu de test:

1. calculezi distanta fata de toate exemplele de antrenare;
2. alegi cei mai apropiati k vecini;
3. la clasificare, iei eticheta majoritara;
4. la regresie, iei media etichetelor vecinilor.

14.2 Distanțe

$$L_1(x, y) = \sum_i |x_i - y_i|.$$

$$L_2(x, y) = \sqrt{\sum_i (x_i - y_i)^2}.$$

$$L_p(x, y) = \left(\sum_i |x_i - y_i|^p \right)^{1/p}.$$

14.3 Efectul lui k

- $k = 1$: model foarte flexibil, risc mare de overfitting;
- k mare: frontiera devine mai neteda, risc de underfitting;
- k se alege pe validare/cross-validation.

15 Regresie liniara si Ridge

15.1 Regresie liniara

Model:

$$\hat{y} = w^T x + b.$$

Pierdere MSE:

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2.$$

15.2 Ridge

Adauga regularizare L2:

$$L(w) = \sum_i (\hat{y}_i - y_i)^2 + \alpha \|w\|_2^2.$$

Efect:

- ponderile sunt tinute mici;
- scade variance;
- ajuta la overfitting si la probleme prost puse.

15.3 Lasso

Adauga regularizare L1:

$$L(w) = \sum_i (\hat{y}_i - y_i)^2 + \alpha \|w\|_1.$$

Lasso poate produce ponderi 0, deci selectie de trasaturi.

16 Mini-exercitii tip examen, rezolvate

16.1 Exercitiul 1 - SVM soft margin

Intrebare. Ce variabila permite unui SVM sa accepte incalcari ale marginii?

Rezolvare. Variabila slack ξ_i . Ea intra in constrangerea:

$$y_i(w^T x_i + b) \geq 1 - \xi_i.$$

Daca $\xi_i > 0$, exemplul incalca marginea. Daca $\xi_i \geq 1$, poate fi gresit clasificat.

16.2 Exercitiul 2 - BatchNorm cu batch de un exemplu

Intrebare. Ce se intampla cu un batch format dintr-un singur exemplu?

Rezolvare. Pentru fiecare dimensiune, media este valoarea acelei dimensiuni si varianta este 0. Dupa normalizare, toate dimensiunile devin 0. Norma exemplului devine 0.

16.3 Exercitiul 3 - Kendall Tau

Intrebare. Pentru $n = 5$, cate perechi verifici?

Rezolvare.

$$\binom{5}{2} = 10.$$

Pentru fiecare pereche verifici daca ordinea reala coincide cu ordinea prezisa.

16.4 Exercitiul 4 - Retele pentru XOR

Intrebare. De ce un neuron liniar nu rezolva XOR, dar un strat ascuns poate?

Rezolvare. XOR nu este liniar separabil. Un singur hiperplan nu poate separa punctele pozitive de cele negative. Un strat ascuns cu activari neliniare poate construi doua sau mai multe regiuni de decizie, deci poate rezolva XOR.

16.5 Exercitiul 5 - Maxout pentru $|x_1 - x_3|$

Intrebare. Scribe un maxout care da 1 cand $x_1 \neq x_3$ si 0 cand $x_1 = x_3$, pentru valori binare.

Rezolvare.

$$f(x) = \max(x_1 - x_3, x_3 - x_1) = |x_1 - x_3|.$$

Pentru valori binare, acesta este exact XOR intre x_1 si x_3 .

17 Fisa de memorat inainte de examen

SVM

- Hiperplan: $w^T x + b = 0$.
- Soft margin: ξ_i permite incalcari.
- C mare = mai strict pe train; C mic = margine mai mare, accepta greseli.

Kernel

- Kernel = produs scalar implicit: $K(x, y) = \langle \phi(x), \phi(y) \rangle$.
- RBF: $\exp(-\gamma \|x - y\|^2)$.
- String kernel: numara n -grame comune.
- Suma si produsul de kerneluri valide sunt kerneluri valide, dar doar daca termenii sunt kerneluri valide pe aceleasi argumente.

Normalizare

- L2: $x' = x / \|x\|_2$.
- In $\mathbb{R}^3$, L2 proiecteaza pe sfera unitate.
- BatchNorm cu un exemplu: output normalizat 0.

Activari

- Sigmoid: $(0, 1)$, saturare.
- Tanh: $[-1, 1]$, medie zero, saturare.
- ReLU: $\max(0, x)$, poate produce dead ReLU.
- Maxout: $\max_j(w_j^T x + b_j)$, neliniar, nu se satureaza.

Optimizare

- GD: $W \leftarrow W - \eta \nabla L$.
- Momentum: $v \leftarrow \mu v - \eta \nabla L$, $W \leftarrow W + v$.
- $\mu = 0$ este SGD standard; $\mu = 1$ este instabil/divergent.

Retele neuronale

- Neliniar separabil: ai nevoie de strat ascuns + activare neliniara.
- Fara activare neliniara, mai multe straturi liniare raman o transformare liniara.
- Dead ReLU apare cand neuronul este mereu pe zona negativa.

Evaluare

- Eroarea empirica aproximeaza eroarea de generalizare cand $n \rightarrow \infty$.
- Kendall Tau: $\tau = (P - Q) / \binom{n}{2}$.
- Bias mare = underfitting; variance mare = overfitting.

18 Surse folosite

Materialul a fost construit pe baza cursurilor si laboratoarelor incarcate: Curs 2 pentru erori, bias/variance si Kendall Tau; Curs 4 si Laboratorul 4 pentru SVM, kerneluri, string kernels si normalizare; Curs 5-6 si Laboratorul 6 pentru retele neuronale, activari, Batch-Norm, backpropagation, SGD si momentum; Laboratoarele 2, 3 si 5 pentru Naive Bayes, KNN si regresie Ridge.

A Modelul din poza - transcriere scurta a temelor

Modelul verifica urmatoarele teme:

1. SVM soft-margin si variabila ξ .
2. Batch normalization pe batch de dimensiune 1.
3. Identitati cu tanh, ReLU si sigmoid.
4. String kernel cu biti de prezenta si 2-grame.
5. Operatii valide cu functii kernel.
6. RBF ca functie bazata pe distanta L2.
7. Momentum in SGD.
8. Conditia ca o retea sa invete date neliniar separabile.
9. Eroarea empirica si eroarea de generalizare.
10. Kendall Tau.
11. Neuron ReLU, dead ReLU si Maxout.
12. Normalizare L2 si separare liniara pe sfera.